

library. If we hadn't tried writing out some of the laws of our API, we may not have discovered the thread resource leak in our first implementation until much later.

In general, there are multiple approaches you can consider when choosing laws for your API. You can think about your conceptual model, and reason from there to postulate laws that should hold. You can also just *invent* laws you think might be useful or instructive (like we did with our `fork` law), and see if it's possible and sensible to ensure that they hold for your model. And lastly, you can look at your *implementation* and come up with laws you expect to hold based on that.<sup>16</sup>

## 7.5 *Refining combinators to their most general form*

Functional design is an iterative process. After you write down your API and have at least a prototype implementation, try using it for progressively more complex or realistic scenarios. Sometimes you'll find that these scenarios require new combinators. But before jumping right to implementation, it's a good idea to see if you can refine the combinator you need to *its most general form*. It may be that what you need is just a specific case of some more general combinator.

### About the exercises in this section

The exercises and answers in this section use our original simpler (blocking) representation of `Par[A]`. If you'd like to work through the exercises and answers using the non-blocking implementation we developed in the previous section instead, see the file `Nonblocking.scala` in both the `exercises` and `answers` projects.

Let's look at an example of this. Suppose we want a function to choose between two forking computations based on the result of an initial computation:

```
def choice[A](cond: Par[Boolean])(t: Par[A], f: Par[A]): Par[A]
```

This constructs a computation that proceeds with `t` if `cond` results in `true`, or `f` if `cond` results in `false`. We can certainly implement this by blocking on the result of the `cond`, and then using this result to determine whether to run `t` or `f`. Here's a simple blocking implementation:<sup>17</sup>

```
def choice[A](cond: Par[Boolean])(t: Par[A], f: Par[A]): Par[A] =
  es =>
    if (run(es)(cond).get) t(es) ← Notice we are blocking
    else f(es)                   on the result of cond.
```

But before we call ourselves good and move on, let's think about this combinator a bit. What is it doing? It's running `cond` and then, when the result is available, it runs either `t` or `f`. This seems reasonable, but let's see if we can think of some variations to get at

<sup>16</sup> This last way of generating laws is probably the weakest, since it can be too easy to just have the laws reflect the implementation, even if the implementation is buggy or requires all sorts of unusual side conditions that make composition difficult.

<sup>17</sup> See `Nonblocking.scala` in the chapter code for the non-blocking implementation.

the essence of this combinator. There's something rather arbitrary about the use of `Boolean` here, and the fact that we're only selecting among *two* possible parallel computations, `t` and `f`. Why just two? If it's useful to be able to choose between two parallel computations based on the results of a first, it should be certainly be useful to choose between  $N$  computations:

```
def choiceN[A] (n: Par[Int]) (choices: List[Par[A]]): Par[A]
```

Let's say that `choiceN` runs `n`, and then uses that to select a parallel computation from `choices`. This is a bit more general than `choice`.



### EXERCISE 7.11

Implement `choiceN` and then `choice` in terms of `choiceN`.

Note what we've done so far. We've refined our original combinator, `choice`, to `choiceN`, which turns out to be more general, capable of expressing `choice` as well as other use cases not supported by `choice`. But let's keep going to see if we can refine `choice` to an even more general combinator.



### EXERCISE 7.12

There's still something rather arbitrary about `choiceN`. The choice of `List` seems overly specific. Why does it matter what sort of container we have? For instance, what if, instead of a list of computations, we have a `Map` of them:<sup>18</sup>

```
def choiceMap[K,V] (key: Par[K]) (choices: Map[K,Par[V]]): Par[V]
```

If you want, stop reading here and see if you can come up with a new and more general combinator in terms of which you can implement `choice`, `choiceN`, and `choiceMap`.

The `Map` encoding of the set of possible choices feels overly specific, just like `List`. If we look at our implementation of `choiceMap`, we can see we aren't really using much of the API of `Map`. Really, the `Map[A,Par[B]]` is used to provide a function, `A => Par[B]`. And now that we've spotted that, looking back at `choice` and `choiceN`, we can see that for `choice`, the pair of arguments was just being used as a function of type `Boolean => Par[A]` (where the `Boolean` selects one of the two `Par[A]` arguments), and for `choiceN` the list was just being used as a function of type `Int => Par[A]`!

<sup>18</sup> `Map[K,V]` (API link: <http://mng.bz/eZ4l>) is a purely functional data structure in the Scala standard library. It associates keys of type `K` with values of type `V` in a one-to-one relationship, and allows us to look up the value by the associated key.

Let's make a more general signature that unifies them all:

```
def chooser[A,B] (pa: Par[A]) (choices: A => Par[B]): Par[B]
```



### EXERCISE 7.13

Implement this new primitive `chooser`, and then use it to implement `choice` and `choiceN`.

Whenever you generalize functions like this, take a critical look at your generalized function when you're finished. Although the function may have been motivated by some specific use case, the signature and implementation may have a more general meaning. In this case, `chooser` is perhaps no longer the most appropriate name for this operation, which is actually quite general—it's a parallel computation that, when run, will run an initial computation whose result is used to determine a second computation. Nothing says that this second computation needs to even exist before the first computation's result is available. It doesn't need to be stored in a container like `List` or `Map`. Perhaps it's being *generated* from whole cloth using the result of the first computation. This function, which comes up often in functional libraries, is usually called `bind` or `flatMap`:

```
def flatMap[A,B] (a: Par[A]) (f: A => Par[B]): Par[B]
```

Is `flatMap` really the most primitive possible function, or can we generalize further? Let's play around with it a bit more. The name `flatMap` is suggestive of the fact that this operation could be decomposed into two steps: *mapping* `f: A => Par[B]` over our `Par[A]`, which generates a `Par[Par[B]]`, and then *flattening* this nested `Par[Par[B]]` to a `Par[B]`. But this is interesting—it suggests that all we needed to do was add an *even simpler* combinator, let's call it `join`, for converting a `Par[Par[X]]` to `Par[X]` for *any* choice of `X`:

```
def join[A] (a: Par[Par[A]]): Par[A]
```

Again we're just following the types. We have an example that demands a function with the given signature, and so we just bring it into existence. Now that it exists, we can think about what the signature means. We call it `join` since conceptually it's a parallel computation that, when run, will execute the inner computation, wait for it to finish (much like `Thread.join`), and then return its result.



### EXERCISE 7.14

Implement `join`. Can you see how to implement `flatMap` using `join`? And can you implement `join` using `flatMap`?